



Tree Basic

Abdullah All Mamun Siam

Lecturer, Dhaka International University(DIU)

Types of Trees

There are three types of trees –

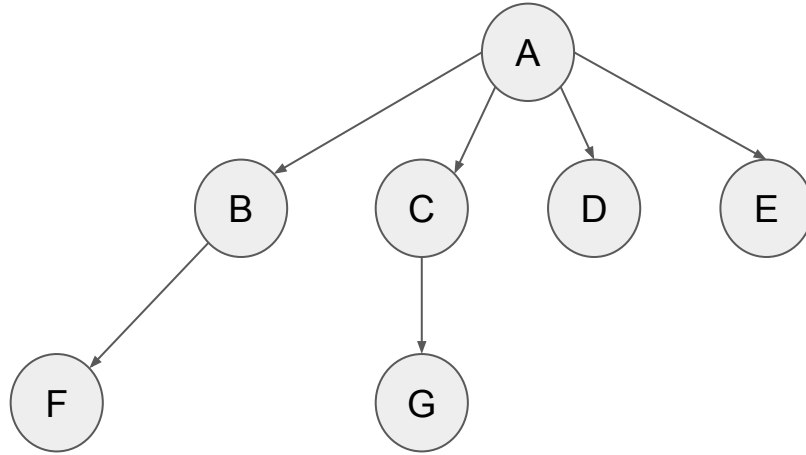
- 1.General Trees

- 2.Binary Trees

- 3.Binary Search Trees

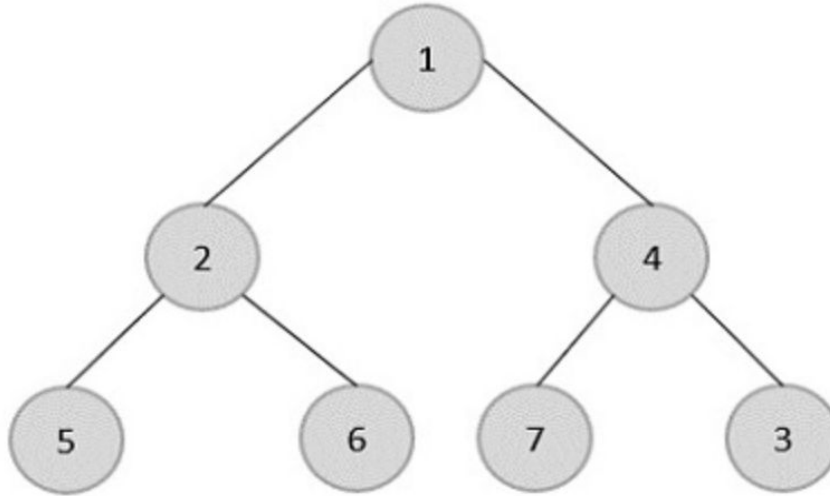
General Tree

A general tree is a hierarchical data structure where each node can have any number of child nodes. Unlike binary trees, which restrict each node to a maximum of two children, general trees impose no such limitation, making them highly versatile for representing complex relationships.



Binary Tree

A binary tree is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. Binary trees are a foundational structure in computer science and are widely used in various applications.



Binary Tree Data Structure

Full Binary Tree, Complete Binary Tree, Perfect Binary Tree

Full Binary Tree

A full binary tree is a binary tree type where every node has either 0 or 2 child nodes.

Complete Binary Tree

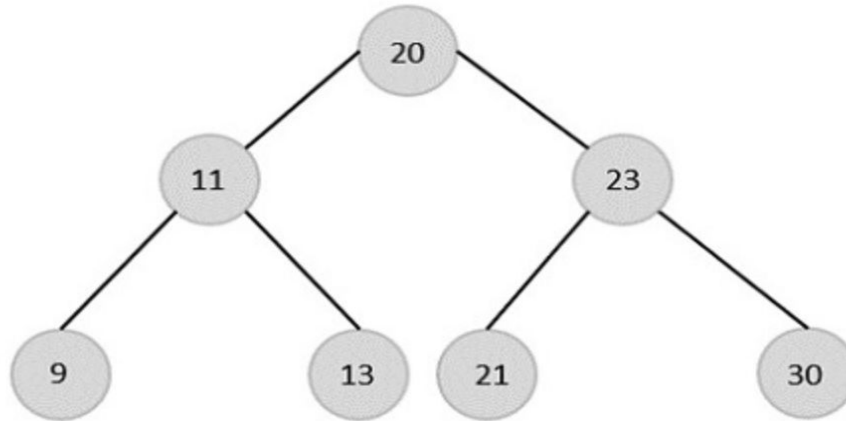
A complete binary tree is a binary tree type where all the leaf nodes must be on the same level. However, root and internal nodes in a complete binary tree can either have 0, 1 or 2 child nodes.

Perfect Binary Tree

A perfect binary tree is a binary tree type where all the leaf nodes are on the same level and every node except leaf nodes have 2 children.

Binary Search Trees

The data in the Binary Search Trees (BST) is always stored in such a way that the values in the left subtree are always less than the values in the root node and the values in the right subtree are always greater than the values in the root node, i.e. $\text{left subtree} < \text{root node} \leq \text{right subtree}$.



Binary Search Tree Data Structure

Advantages of BST

- 1.Binary Search Trees are more efficient than Binary Trees since time complexity for performing various operations reduces.
- 2.Since the order of keys is based on just the parent node, searching operation becomes simpler.
- 3.The alignment of BST also favors Range Queries, which are executed to find values existing between two keys. This helps in the Database Management System.
- 4.Insertions and deletions can be done efficiently.

Disadvantages of Binary Search Trees (BST)

Inefficient Performance in the Worst Case

In the worst case, if the tree becomes unbalanced (e.g., inserting nodes in sorted order), the BST essentially degenerates into a linked list, leading to poor performance.

For example, searching, inserting, and deleting nodes would have a time complexity of $O(n)$ instead of the expected $O(\log n)$.

Example:

1,2,3,4,5,6,7

In this case, searching for a value or inserting a new node will take linear time, which is inefficient for large datasets.

Disadvantages of Binary Search Trees (BST)

Requires Balancing to Maintain Efficiency

To maintain efficient $O(\log n)$ time complexity for search, insertion, and deletion, the BST needs to remain balanced. However, a simple BST doesn't automatically self-balance, and maintaining balance requires additional algorithms, such as AVL trees or Red-Black trees, which involve extra computation.

Memory Overhead

For each node, a BST requires memory for left and right child pointers in addition to the data stored in the node. For very large datasets, this overhead can add up, especially if nodes are sparse or if the tree is not well-balanced.

Disadvantages of Binary Search Trees (BST)

Difficult to Handle Duplicate Values

A standard BST does not handle duplicate values efficiently. To handle duplicates, one needs to modify the tree's insert operation, which can complicate the logic and potentially reduce performance.

Balanced Binary Search Tree (BBST)

Balanced Binary Search Tree (BBST) means that the difference in heights between the left and right subtrees of any node must be -1, 0, or 1.

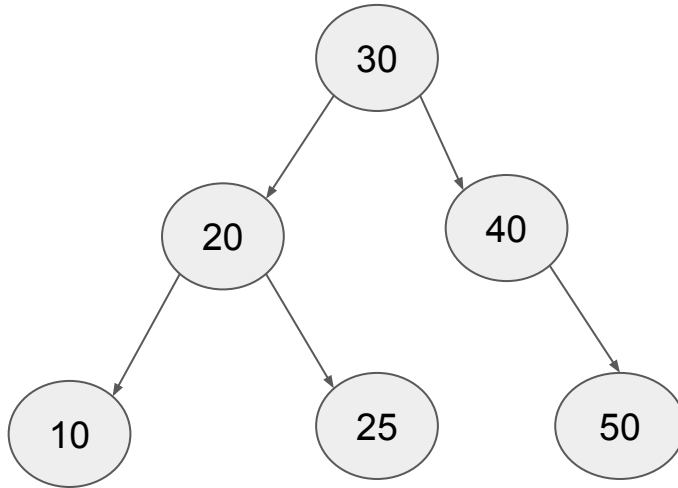
$$|m-n| \leq 1$$

Where:

m is the height of the left subtree.

n is the height of the right subtree.

Example of Balanced Binary Search Tree (BBST)



Example of Balanced Binary Search Tree (BBST)

Root Node: 30

- Left subtree height = 2 (height of node 20)
- Right subtree height = 2 (height of node 40)
- Balance factor: $|2 - 2| = 0$ (balanced)

Node 20:

- Left subtree height = 1 (height of node 10)
- Right subtree height = 1 (height of node 25)
- Balance factor: $|1 - 1| = 0$ (balanced)

Node 40:

- Left subtree height = 0 (no left child)
- Right subtree height = 1 (height of node 50)
- Balance factor: $|0 - 1| = 1$ (balanced)

Example of Balanced Binary Search Tree (BBST)

Node 10:

- No children, so the height is 0.
- Balance factor is not relevant here as it is a leaf node.

Node 25:

- No children, so the height is 0.
- Balance factor is not relevant here as it is a leaf node.

Node 50:

- No children, so the height is 0.
- Balance factor is not relevant here as it is a leaf node.